

In [440]:

```
import os
import requests
from dotenv import load_dotenv
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

```
load_dotenv()
api_key = os.getenv("CENSUS_API_KEY")
```

In [441]:

```
acs_url = "https://api.census.gov/data/2024/acs/acs5"

acs_variables = [
    "NAME",
    "B28002_001E", # Total households
    "B28002_004E", # Households with broadband subscription
    "B19013_001E", # Median household income
    "B25064_001E", # Median gross rent
    "B17001_001E", # Population for whom poverty status is determined
    "B17001_002E", # Population below poverty level
    "B15003_001E", # Population 25+ with educational attainment data
    "B15003_022E", # Bachelor's degree
    "B15003_023E", # Master's degree
    "B15003_024E", # Professional degree
    "B15003_025E", # Doctorate degree
    "B23025_003E", # Civilian labor force
    "B23025_005E" # Unemployed
]

params = {
    "get": ",".join(acs_variables),
    "for": "county:*",
    "in": "state:37",
    "key": api_key
}
response = requests.get(acs_url, params=params)

response.status_code
```

Out[441]:

200

In [442]:

```
data = response.json()

acs_df = pd.DataFrame(data[1:], columns=data[0])

acs_df.head()
```

Out[442]:

	NAME	B28002_001E	B28002_004E	B19013_001E	B25064_001E	B17001_001E	B17001_002E	B1
0	Alamance County,	70062	63189	65651	1064	171342	22876	

	NAME	B28002_001E	B28002_004E	B19013_001E	B25064_001E	B17001_001E	B17001_002E	B1
	North Carolina							
1	Alexander County, North Carolina	14126	12023	65354	764	34657	4357	
2	Alleghany County, North Carolina	5177	4170	47172	675	11077	1770	
3	Anson County, North Carolina	8548	7296	47302	814	20635	4483	
4	Ashe County, North Carolina	12201	10399	56866	859	26611	3590	

In [443]:

```
acs_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0    NAME            100 non-null    object
1    B28002_001E      100 non-null    object
2    B28002_004E      100 non-null    object
3    B19013_001E      100 non-null    object
4    B25064_001E      100 non-null    object
5    B17001_001E      100 non-null    object
6    B17001_002E      100 non-null    object
7    B15003_001E      100 non-null    object
8    B15003_022E      100 non-null    object
9    B15003_023E      100 non-null    object
10   B15003_024E      100 non-null    object
11   B15003_025E      100 non-null    object
12   B23025_003E      100 non-null    object
13   B23025_005E      100 non-null    object
14   state            100 non-null    object
15   county           100 non-null    object
dtypes: object(16)
memory usage: 12.6+ KB
```

In [444]:

```
numeric_columns = [
    "B28002_001E",
    "B28002_004E",
    "B19013_001E",
    "B25064_001E",
    "B17001_001E",
    "B17001_002E",
    "B15003_001E",
    "B15003_022E",
```

```

        "B15003_023E",
        "B15003_024E",
        "B15003_025E",
        "B23025_003E",
        "B23025_005E"
    ]

    acs_df[numeric_columns] = acs_df[numeric_columns].apply(
        pd.to_numeric,
        errors="coerce"
    )

```

In [445]:

```
acs_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   NAME            100 non-null   object
1   B28002_001E     100 non-null   int64
2   B28002_004E     100 non-null   int64
3   B19013_001E     100 non-null   int64
4   B25064_001E     100 non-null   int64
5   B17001_001E     100 non-null   int64
6   B17001_002E     100 non-null   int64
7   B15003_001E     100 non-null   int64
8   B15003_022E     100 non-null   int64
9   B15003_023E     100 non-null   int64
10  B15003_024E     100 non-null   int64
11  B15003_025E     100 non-null   int64
12  B23025_003E     100 non-null   int64
13  B23025_005E     100 non-null   int64
14  state           100 non-null   object
15  county          100 non-null   object
dtypes: int64(13), object(3)
memory usage: 12.6+ KB

```

In [446]:

```
acs_df.isna().sum()
```

Out[446]:

```
NAME          0
B28002_001E   0
B28002_004E   0
B19013_001E   0
B25064_001E   0
B17001_001E   0
B17001_002E   0
B15003_001E   0
B15003_022E   0
B15003_023E   0
B15003_024E   0
B15003_025E   0
B23025_003E   0
B23025_005E   0
state         0
county        0
dtype: int64
```

In [447]:

```
acs_df.shape
acs_df.columns
acs_df.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 100 entries, 0 to 99

Data columns (total 16 columns):

#	Column	Non-Null Count	Dtype
0	NAME	100 non-null	object
1	B28002_001E	100 non-null	int64
2	B28002_004E	100 non-null	int64
3	B19013_001E	100 non-null	int64
4	B25064_001E	100 non-null	int64
5	B17001_001E	100 non-null	int64
6	B17001_002E	100 non-null	int64
7	B15003_001E	100 non-null	int64
8	B15003_022E	100 non-null	int64
9	B15003_023E	100 non-null	int64
10	B15003_024E	100 non-null	int64
11	B15003_025E	100 non-null	int64
12	B23025_003E	100 non-null	int64
13	B23025_005E	100 non-null	int64
14	state	100 non-null	object
15	county	100 non-null	object

dtypes: int64(13), object(3)

memory usage: 12.6+ KB

In [448]:

```
acs_df["broadband_access"] = (
    acs_df["B28002_004E"] /
    acs_df["B28002_001E"]
) * 100

acs_df["poverty_rate"] = (
    acs_df["B17001_002E"] /
    acs_df["B17001_001E"]
) * 100

acs_df["bachelors_or_higher"] = (
```

```

    acs_df["B15003_022E"] +
    acs_df["B15003_023E"] +
    acs_df["B15003_024E"] +
    acs_df["B15003_025E"]
) / acs_df["B15003_001E"] * 100

acs_df["unemployment_rate"] = (
    acs_df["B23025_005E"] /
    acs_df["B23025_003E"]
) * 100

```

In [449]:

```

acs_df[
    [
        "NAME",
        "broadband_access",
        "B19013_001E",
        "B25064_001E",
        "poverty_rate",
        "bachelors_or_higher",
        "unemployment_rate"
    ]
].head()

```

Out[449]:

	NAME	broadband_access	B19013_001E	B25064_001E	poverty_rate	bachelors_or_higher	unempr
0	Alamance County, North Carolina	90.190117	65651	1064	13.351076	28.715104	
1	Alexander County, North Carolina	85.112558	65354	764	12.571775	16.661688	
2	Alleghany County, North Carolina	80.548580	47172	675	15.979056	23.577423	
3	Anson County, North Carolina	85.353299	47302	814	21.725224	9.908961	
4	Ashe County, North Carolina	85.230719	56866	859	13.490662	25.938286	

In [450]:

```

acs_df = acs_df.rename(columns={
    "NAME": "county_name",
    "county": "county_fips",
    "B19013_001E": "median_household_income",
    "B25064_001E": "median_gross_rent"
})

acs_df[[

```

```

    "county_name",
    "county_fips",
    "state",
    "broadband_access",
    "median_household_income",
    "median_gross_rent",
    "poverty_rate",
    "bachelors_or_higher",
    "unemployment_rate"
]] .head()

```

Out[450]:

	county_name	county_fips	state	broadband_access	median_household_income	median_gross_rent
0	Alamance County, North Carolina	001	37	90.190117	65651	1064
1	Alexander County, North Carolina	003	37	85.112558	65354	764
2	Alleghany County, North Carolina	005	37	80.548580	47172	675
3	Anson County, North Carolina	007	37	85.353299	47302	814
4	Ashe County, North Carolina	009	37	85.230719	56866	859

In [451]:

```

rural_url = "https://api.census.gov/data/2020/dec/dhc"

rural_variables = [
    "NAME",
    "P2_001N", # Total population
    "P2_003N"  # Rural population
]

params = {
    "get": ",".join(rural_variables),
    "for": "county:*",
    "in": "state:37",
    "key": api_key
}

response = requests.get(rural_url, params=params)

response.status_code

```

Out[451]:

200

In [452]:

```

#Processing the rural broadband data
rural_data = response.json()

rural_df = pd.DataFrame(

```

```

    rural_data[1:],
    columns=rural_data[0]
)

```

In [453]:

```

#Viewing the information about the rural broadband data
rural_df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   NAME         100 non-null    object
1   P2_001N      100 non-null    object
2   P2_003N      100 non-null    object
3   state        100 non-null    object
4   county       100 non-null    object
dtypes: object(5)
memory usage: 4.0+ KB

```

In [454]:

```

#Renaming the columns of the rural_df DataFrame for clarity
rural_df = rural_df.rename(columns={
    "NAME": "county",
    "P2_001N": "total_population_2020",
    "P2_003N": "rural_population_2020",
    "county": "county_fips"
})

```

In [455]:

```

#Converting the data types of the relevant columns to numeric
rural_df[["total_population_2020", "rural_population_2020"]] = (
    rural_df[["total_population_2020", "rural_population_2020"]]
    .apply(pd.to_numeric, errors="coerce")
)

```

In [456]:

```

#Validating the changes by viewing the information about the rural broadband data again
rural_df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
#   Column              Non-Null Count  Dtype
---  -
0   county              100 non-null    object
1   total_population_2020 100 non-null    int64
2   rural_population_2020 100 non-null    int64
3   state               100 non-null    object
4   county_fips         100 non-null    object
dtypes: int64(2), object(3)
memory usage: 4.0+ KB

```

In [457]:

```

#Calculating the percentage of the rural population in each county
rural_df["rural_percent"] = (
    rural_df["rural_population_2020"] /

```

```
rural_df["total_population_2020"]
) * 100
```

In [458]:

```
#Displaying the first few rows of the rural_df DataFrame to verify the changes
rural_df.head()
```

Out[458]:

	county	total_population_2020	rural_population_2020	state	county_fips	rural_percent
0	Randolph County, North Carolina	144171	86768	37	151	60.184087
1	Person County, North Carolina	39097	29597	37	145	75.701460
2	Pitt County, North Carolina	170243	45713	37	147	26.851618
3	Clay County, North Carolina	11089	11089	37	043	100.000000
4	Chatham County, North Carolina	76285	48850	37	037	64.036180

In [459]:

```
final_df = acs_df.merge(
    rural_df[
        [
            "state",
            "county_fips",
            "total_population_2020",
            "rural_population_2020",
            "rural_percent"
        ]
    ],
    on=["state", "county_fips"],
    how="inner"
)
```

In [460]:

```
acs_df.shape, rural_df.shape, final_df.shape
```

Out[460]:

```
((100, 20), (100, 6), (100, 23))
```

In [461]:

```
final_df.head()
```

Out[461]:

	county_name	B28002_001E	B28002_004E	median_household_income	median_gross_rent	B17001_0
0	Alamance County, North Carolina	70062	63189	65651	1064	171
1	Alexander County, North Carolina	14126	12023	65354	764	34
2	Alleghany County, North	5177	4170	47172	675	11

	county_name	B28002_001E	B28002_004E	median_household_income	median_gross_rent	B17001_0
	Carolina					
3	Anson County, North Carolina	8548	7296	47302	814	20
4	Ashe County, North Carolina	12201	10399	56866	859	26

5 rows × 23 columns

In [462]:

```
# Convert the raw Census variables to numeric values

numeric_columns = [
    "B28002_001E",
    "B28002_004E",
    "B17001_001E",
    "B17001_002E",
    "B15003_001E",
    "B15003_022E",
    "B15003_023E",
    "B15003_024E",
    "B15003_025E",
    "B23025_003E",
    "B23025_005E"
]

final_df[numeric_columns] = final_df[numeric_columns].apply(
    pd.to_numeric,
    errors="coerce"
)

# Convert income and rent to numeric values
final_df[[
    "median_household_income",
    "median_gross_rent"
]] = final_df[[
    "median_household_income",
    "median_gross_rent"
]].apply(pd.to_numeric, errors="coerce")
```

In [463]:

```
# Calculate the derived variables
final_df["broadband_access"] = (
    final_df["B28002_004E"] /
    final_df["B28002_001E"]
) * 100

final_df["poverty_rate"] = (
    final_df["B17001_002E"] /
    final_df["B17001_001E"]
) * 100

final_df["bachelors_or_higher"] = (
    final_df["B15003_022E"] +
    final_df["B15003_023E"] +
```

```

    final_df["B15003_024E"] +
    final_df["B15003_025E"]
) / final_df["B15003_001E"] * 100

final_df["unemployment_rate"] = (
    final_df["B23025_005E"] /
    final_df["B23025_003E"]
) * 100

```

In [464]:

```
final_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 100 entries, 0 to 99
```

```
Data columns (total 23 columns):
```

#	Column	Non-Null Count	Dtype
0	county_name	100 non-null	object
1	B28002_001E	100 non-null	int64
2	B28002_004E	100 non-null	int64
3	median_household_income	100 non-null	int64
4	median_gross_rent	100 non-null	int64
5	B17001_001E	100 non-null	int64
6	B17001_002E	100 non-null	int64
7	B15003_001E	100 non-null	int64
8	B15003_022E	100 non-null	int64
9	B15003_023E	100 non-null	int64
10	B15003_024E	100 non-null	int64
11	B15003_025E	100 non-null	int64
12	B23025_003E	100 non-null	int64
13	B23025_005E	100 non-null	int64
14	state	100 non-null	object
15	county_fips	100 non-null	object
16	broadband_access	100 non-null	float64
17	poverty_rate	100 non-null	float64
18	bachelors_or_higher	100 non-null	float64
19	unemployment_rate	100 non-null	float64
20	total_population_2020	100 non-null	int64
21	rural_population_2020	100 non-null	int64
22	rural_percent	100 non-null	float64

```
dtypes: float64(5), int64(15), object(3)
```

```
memory usage: 18.1+ KB
```

In [465]:

```
final_df.isna().sum()
```

Out[465]:

county_name	0
B28002_001E	0
B28002_004E	0
median_household_income	0
median_gross_rent	0
B17001_001E	0
B17001_002E	0
B15003_001E	0
B15003_022E	0
B15003_023E	0
B15003_024E	0
B15003_025E	0
B23025_003E	0

```
B23025_005E      0
state            0
county_fips      0
broadband_access 0
poverty_rate     0
bachelors_or_higher 0
unemployment_rate 0
total_population_2020 0
rural_population_2020 0
rural_percent    0
dtype: int64
```

In [466]:

```
final_df[[
    "county_name",
    "broadband_access",
    "median_household_income",
    "median_gross_rent",
    "poverty_rate",
    "bachelors_or_higher",
    "unemployment_rate",
    "rural_percent"
]].head()
```

Out[466]:

	county_name	broadband_access	median_household_income	median_gross_rent	poverty_rate	bach
0	Alamance County, North Carolina	90.190117	65651	1064	13.351076	
1	Alexander County, North Carolina	85.112558	65354	764	12.571775	
2	Alleghany County, North Carolina	80.548580	47172	675	15.979056	
3	Anson County, North Carolina	85.353299	47302	814	21.725224	
4	Ashe County, North Carolina	85.230719	56866	859	13.490662	

In [467]:

```
model_df = final_df[[
    "county_name",
    "broadband_access",
    "median_household_income",
    "median_gross_rent",
    "poverty_rate",
    "bachelors_or_higher",
    "unemployment_rate",
    "rural_percent"
]].copy()
```

In [468]:

```
model_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   county_name                          100 non-null    object
1   broadband_access                     100 non-null    float64
2   median_household_income              100 non-null    int64
3   median_gross_rent                    100 non-null    int64
4   poverty_rate                         100 non-null    float64
5   bachelors_or_higher                  100 non-null    float64
6   unemployment_rate                    100 non-null    float64
7   rural_percent                        100 non-null    float64
dtypes: float64(5), int64(2), object(1)
memory usage: 6.4+ KB
```

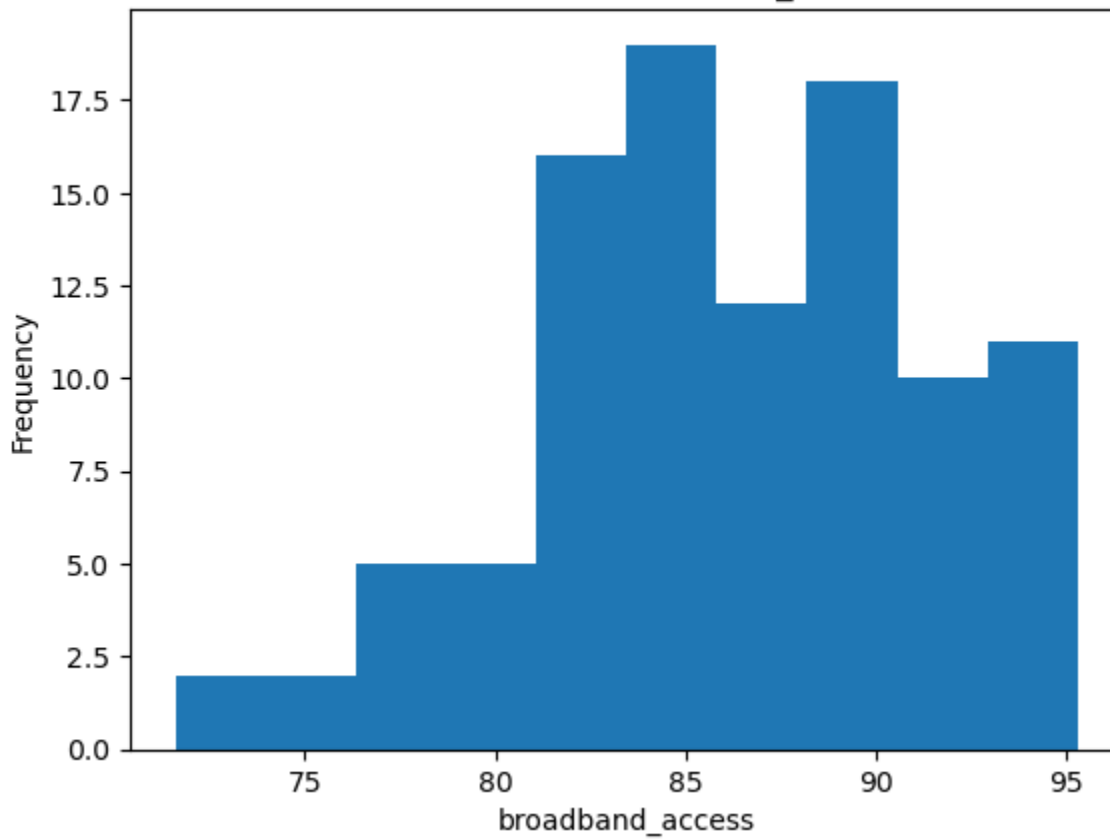
```
In [469]:
#Distribution Analysis - Broadband Access and Predictors (Histograms, Boxplots, and Summ

#Histograms for each variable except county_name
for column in model_df.columns:
    if column != 'county_name':
        plt.hist(model_df[column])
        plt.xlabel(column)
        plt.ylabel('Frequency')
        plt.title(f'Distribution of {column}')
    plt.show()

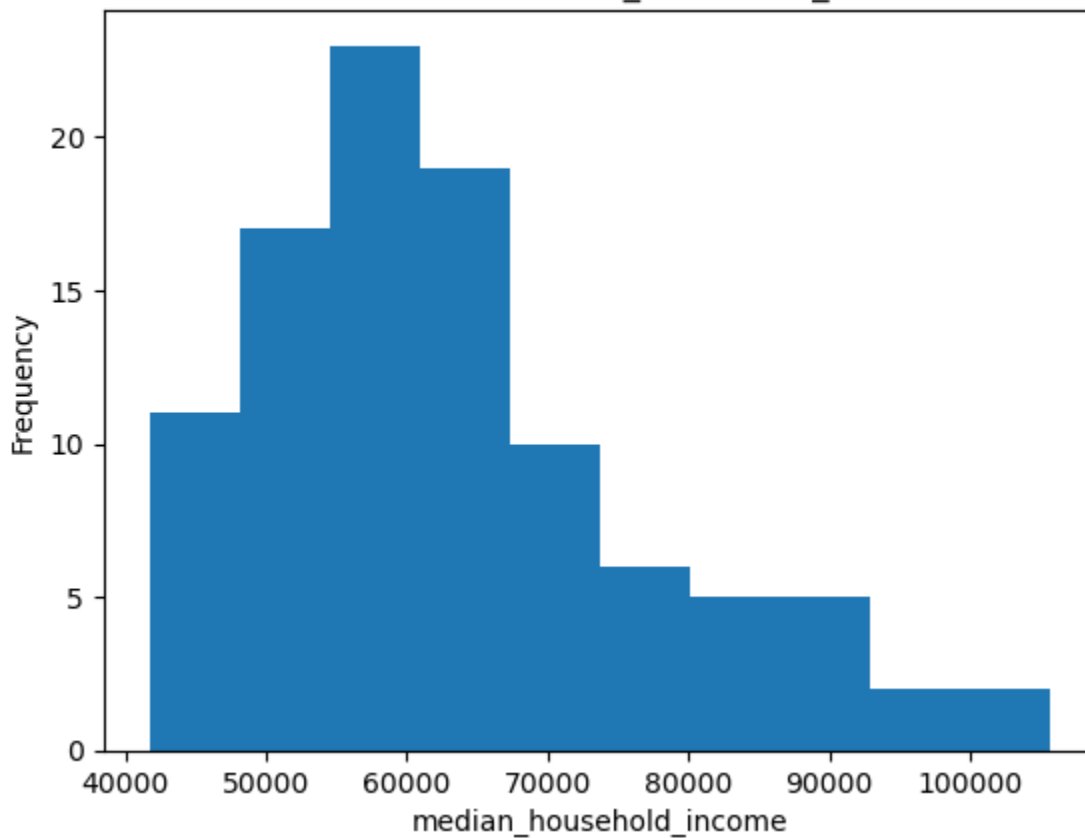
#Boxplots for each variable except county_name
for column in model_df.columns:
    if column != 'county_name':
        plt.boxplot(model_df[column])
        plt.ylabel(column)
        plt.title(f'Boxplot of {column}')
    plt.show()

#Summary Statistics
summary_stats = model_df.describe()
summary_stats
```

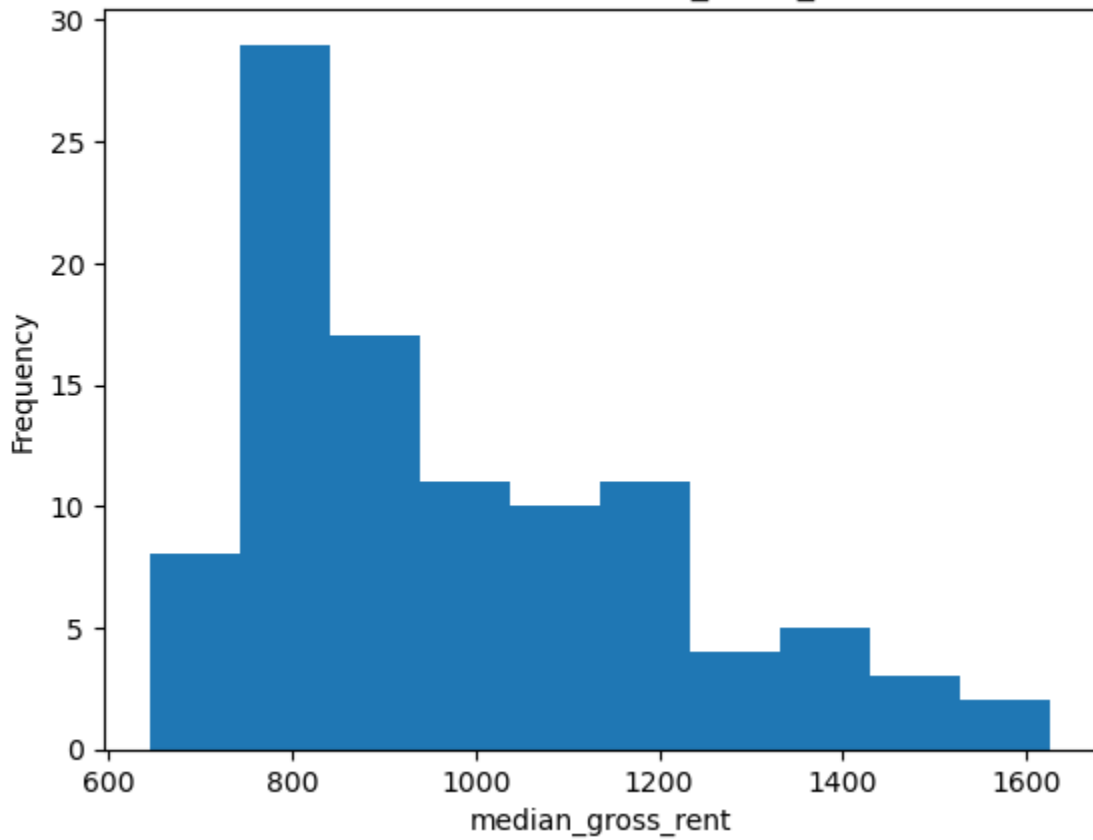
Distribution of broadband_access



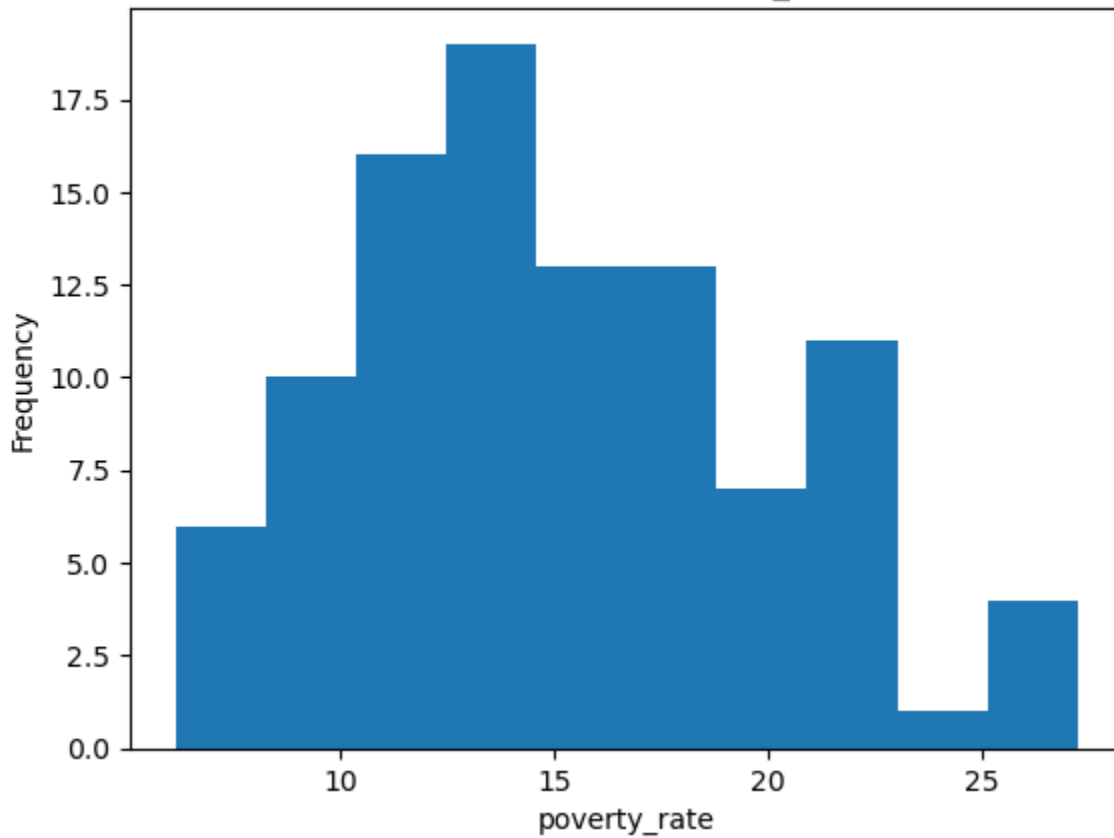
Distribution of median_household_income



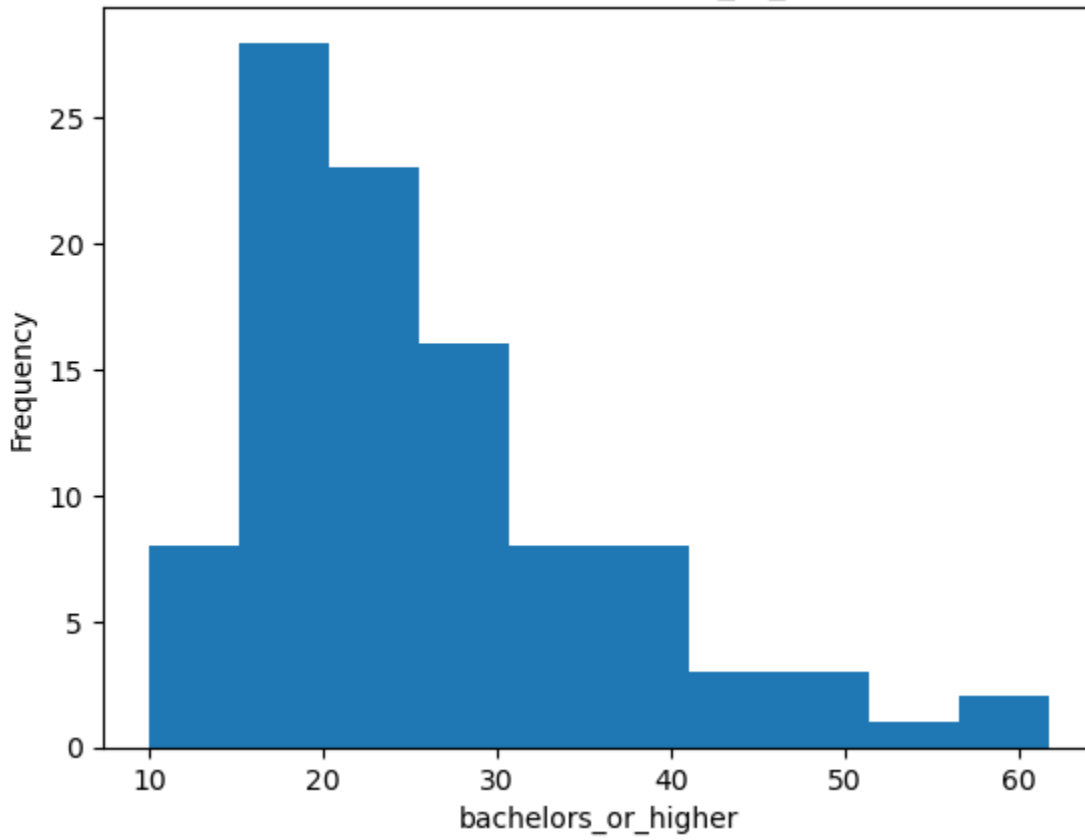
Distribution of median_gross_rent



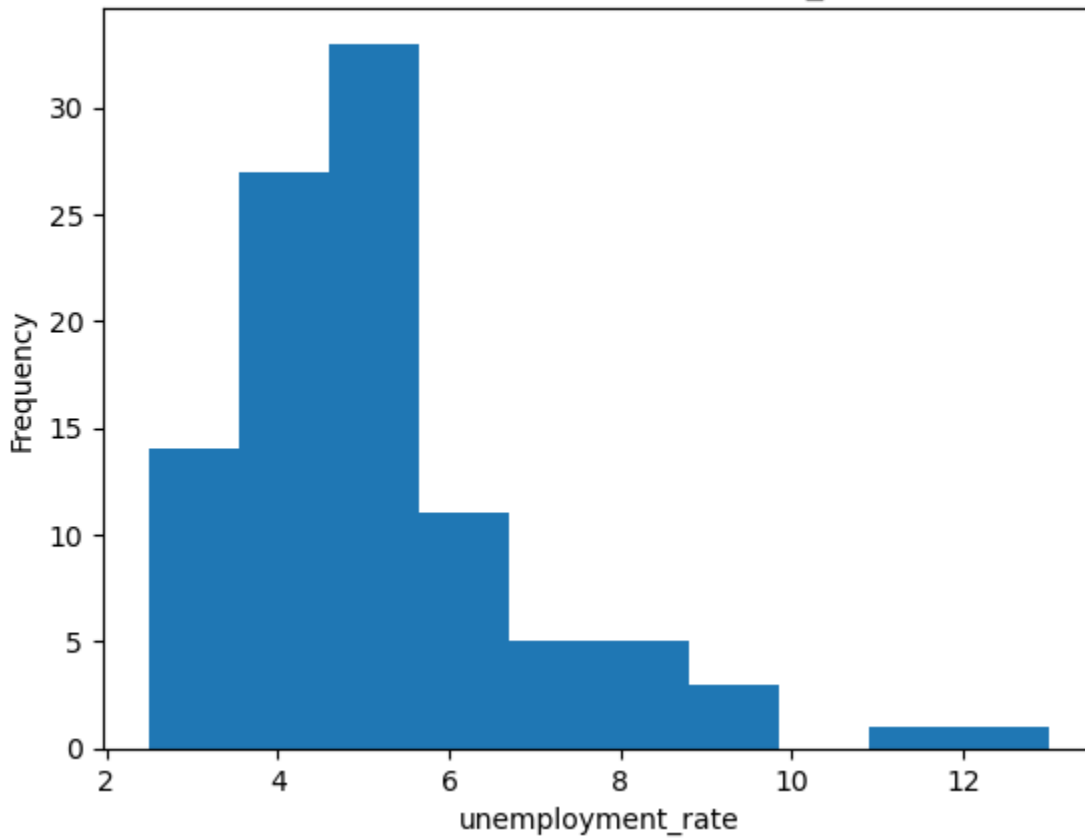
Distribution of poverty_rate



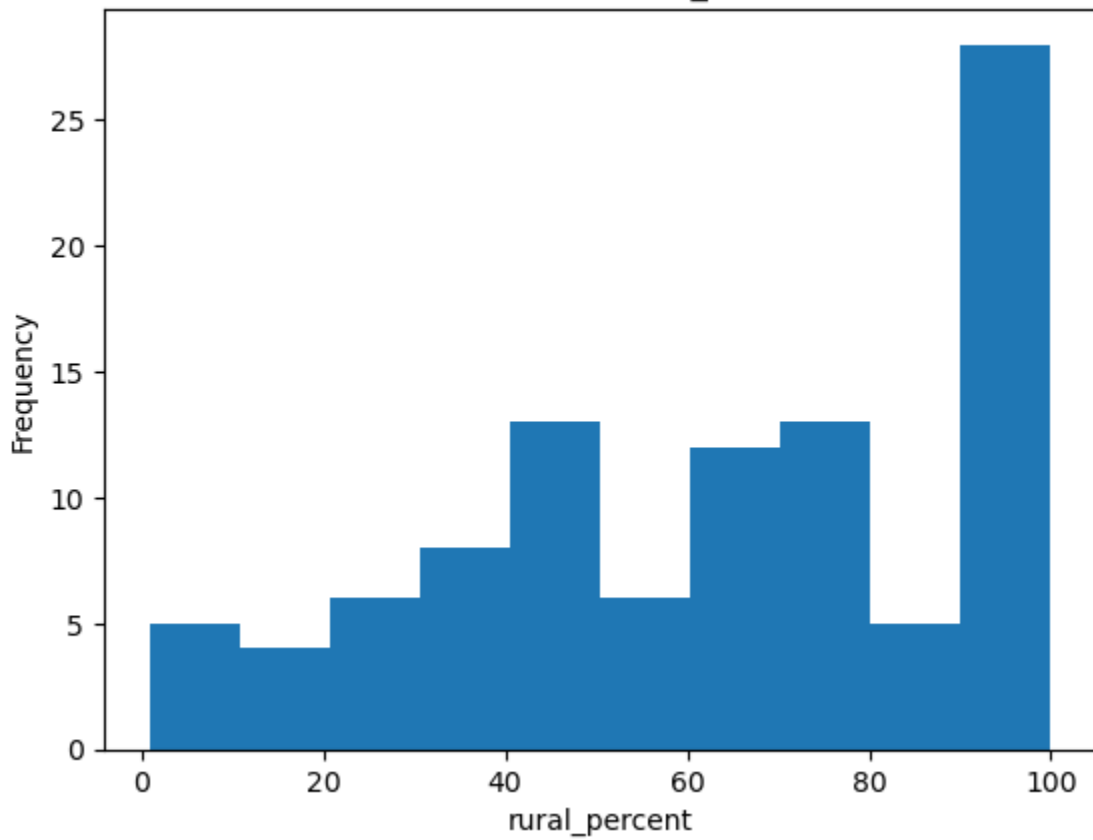
Distribution of bachelors_or_higher



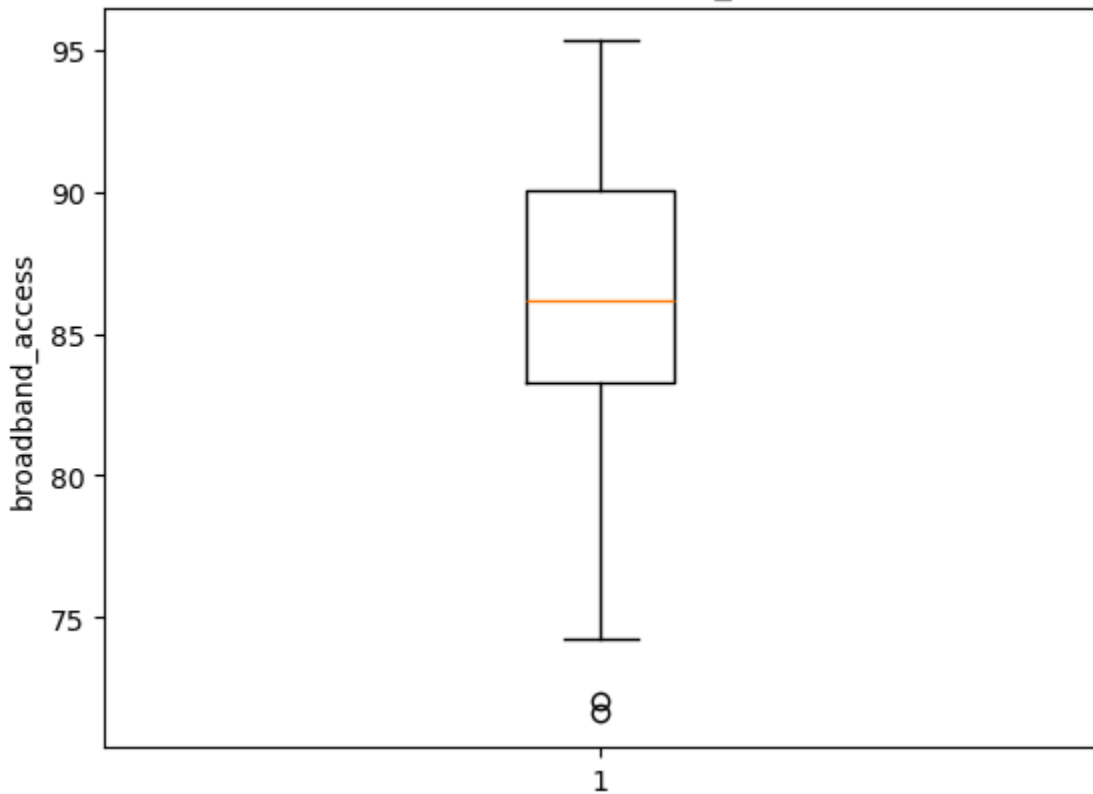
Distribution of unemployment_rate



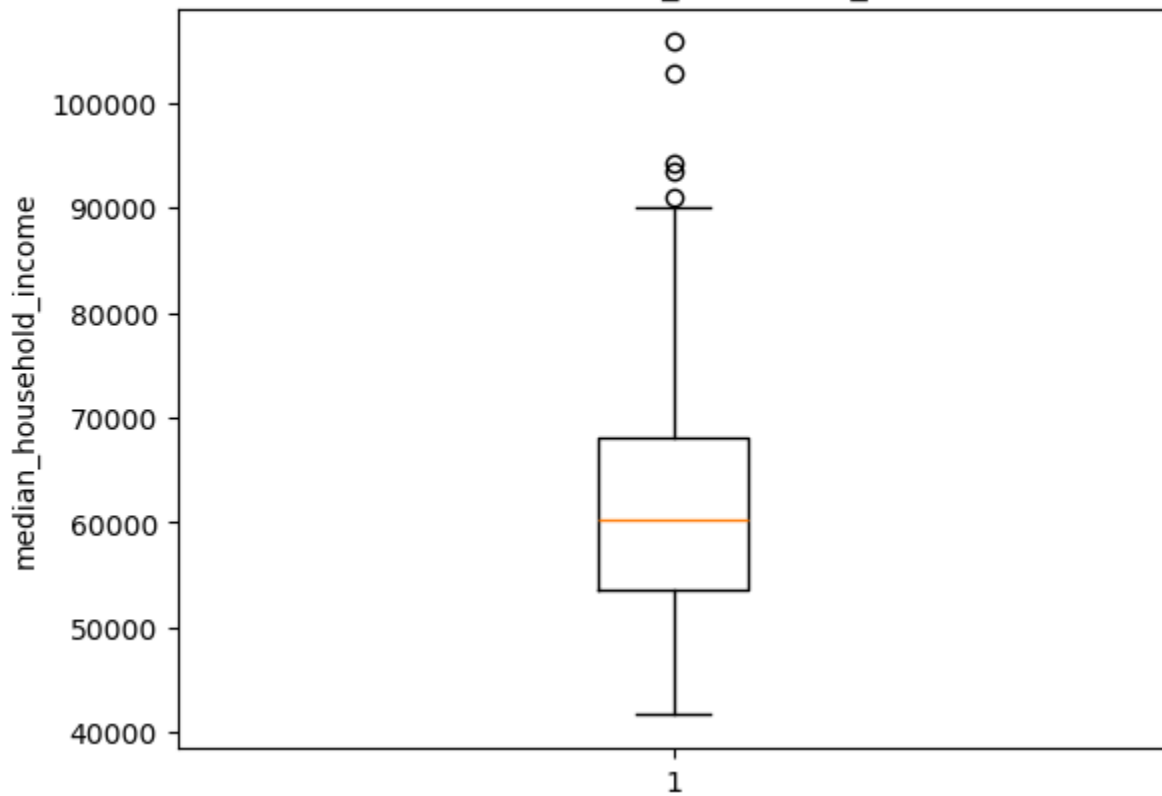
Distribution of rural_percent



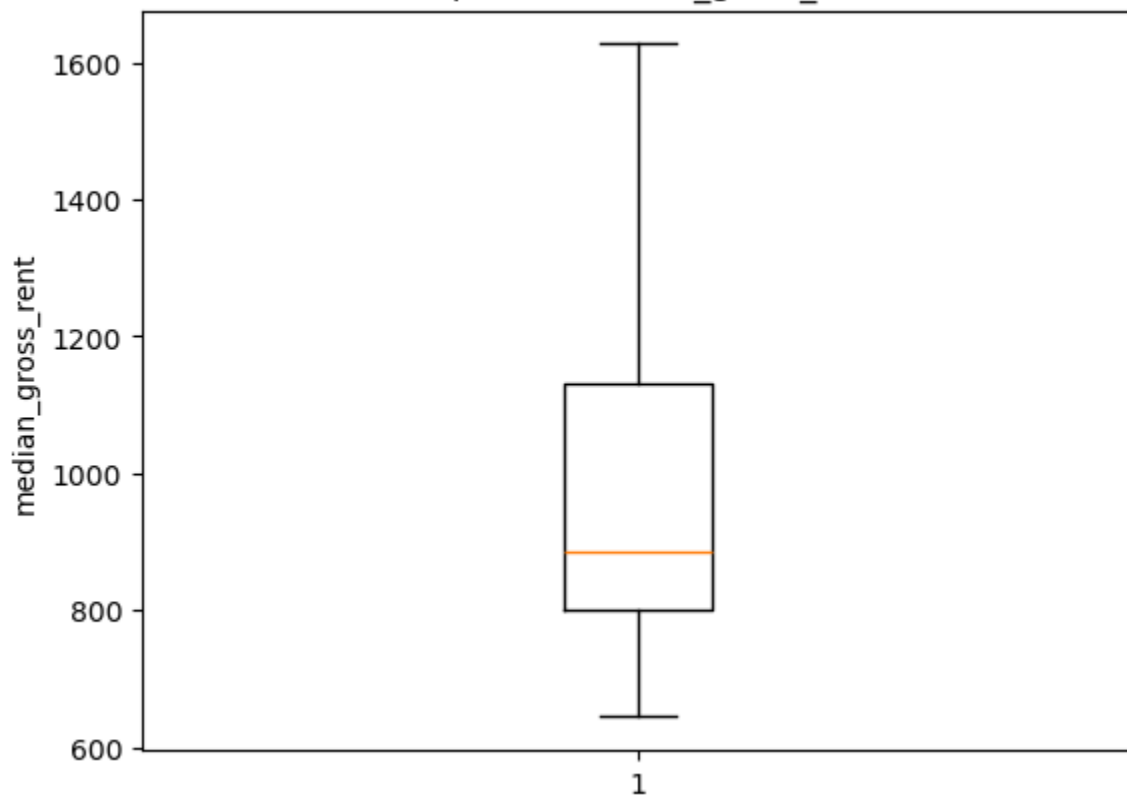
Boxplot of broadband_access



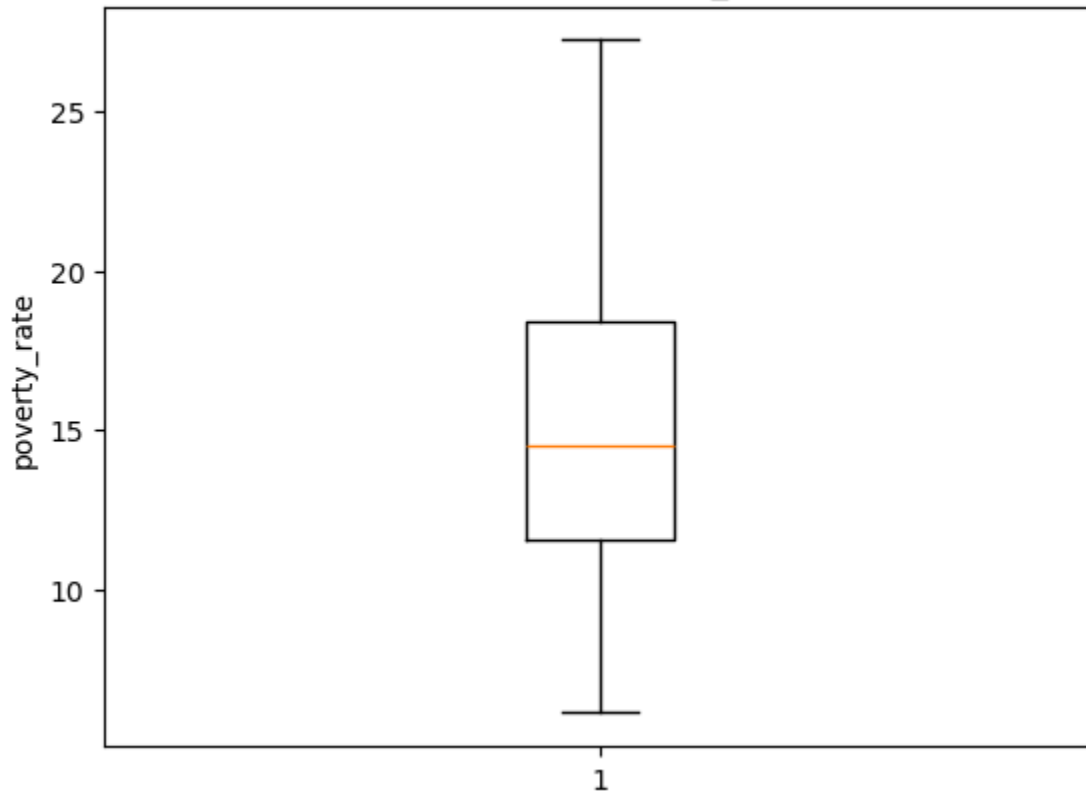
Boxplot of median_household_income



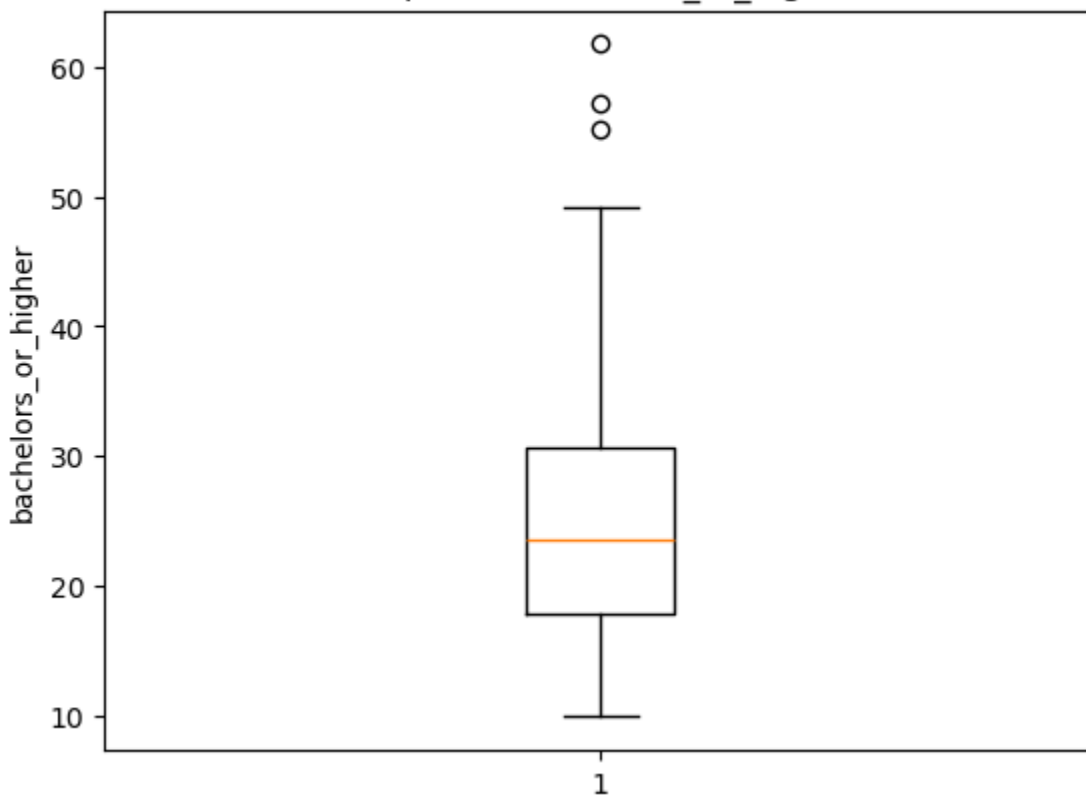
Boxplot of median_gross_rent

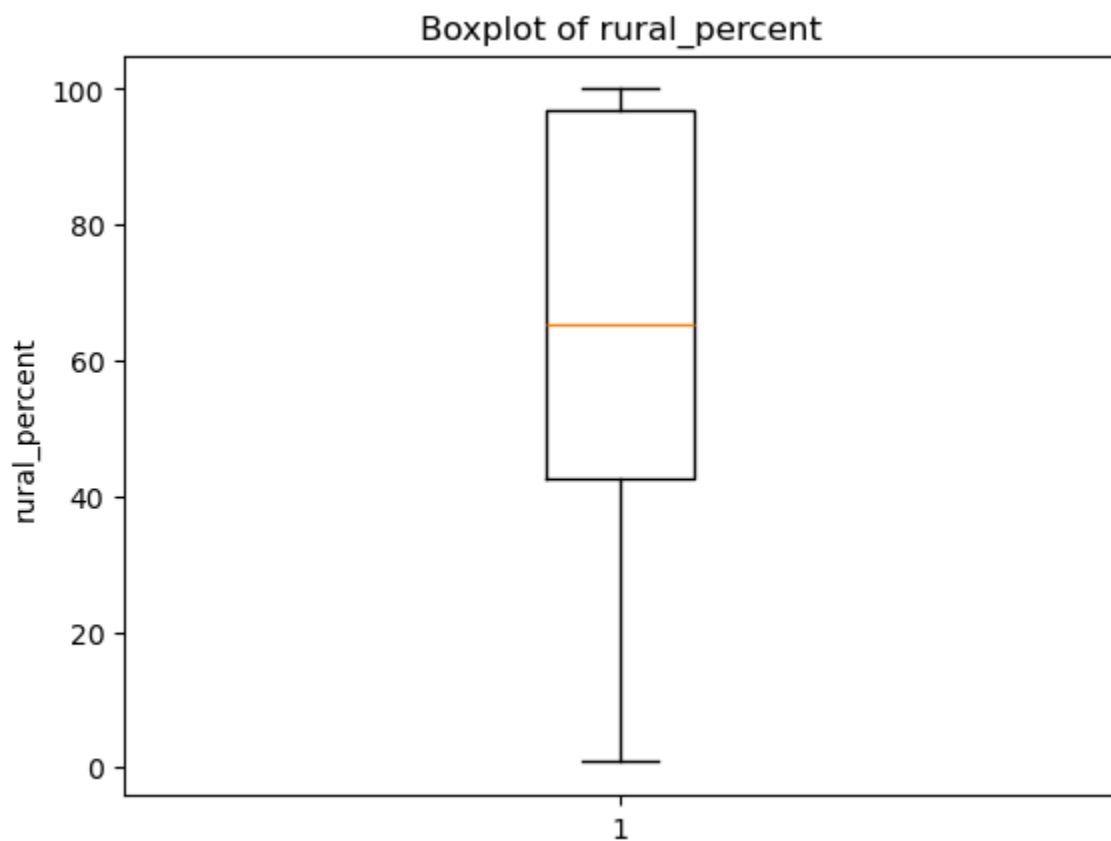


Boxplot of poverty_rate



Boxplot of bachelors_or_higher





Out[469]:

	broadband_access	median_household_income	median_gross_rent	poverty_rate	bachelors_or_hi
count	100.000000	100.000000	100.000000	100.000000	100.00
mean	86.179052	63181.72000	976.69000	15.180208	26.00
std	5.090073	13886.69788	230.90965	4.776834	10.58

	broadband_access	median_household_income	median_gross_rent	poverty_rate	bachelors_or_hi
min	71.608568	41685.00000	644.00000	6.141805	9.90
25%	83.289890	53532.00000	799.75000	11.584002	17.88
50%	86.173709	60229.50000	885.50000	14.547185	23.51
75%	90.051783	68157.75000	1131.00000	18.425979	30.67
max	95.304830	105768.00000	1627.00000	27.228705	61.75

In [470]:

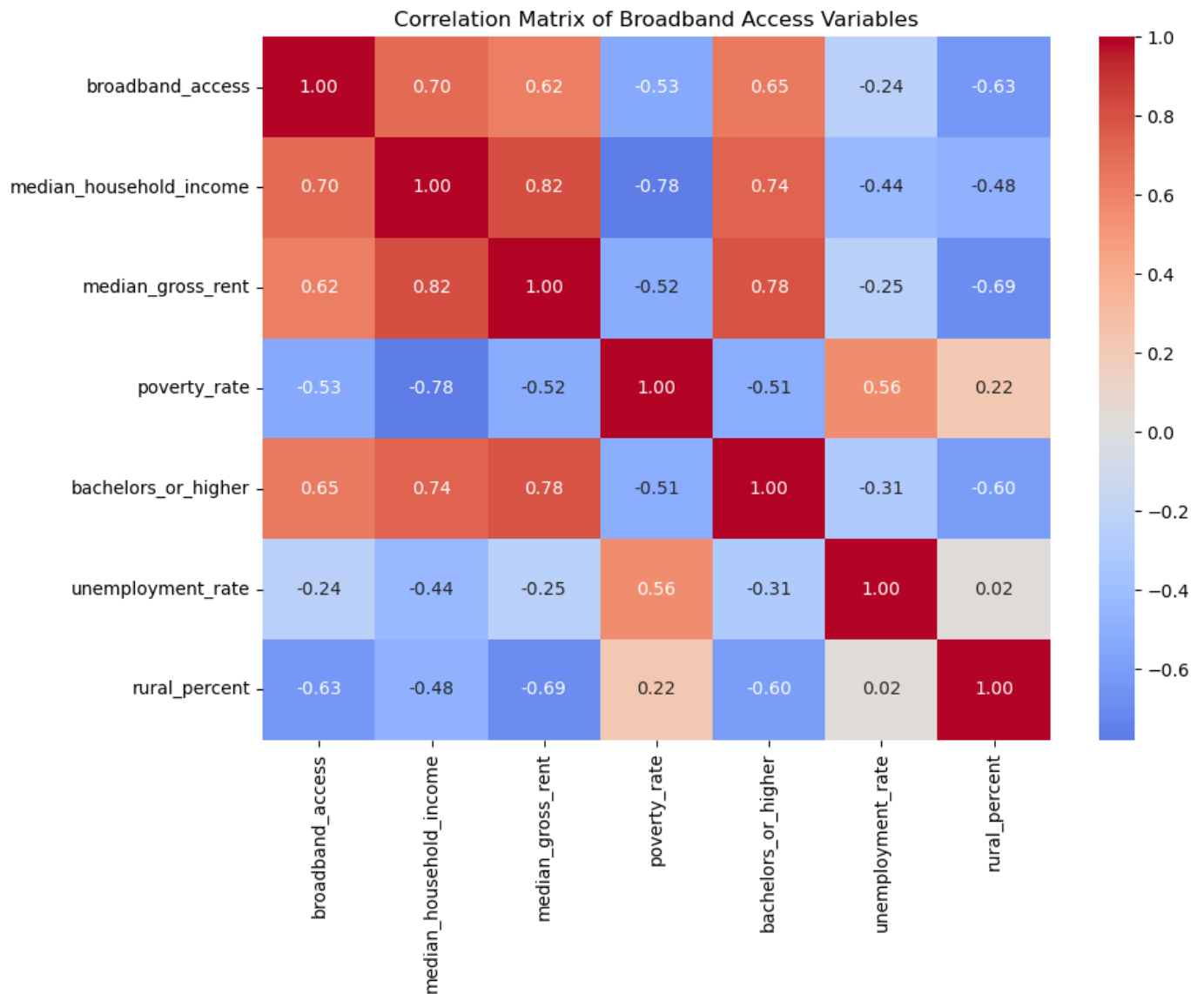
```
correlation_df = model_df.drop(columns="county_name")

correlation_matrix = correlation_df.corr()

plt.figure(figsize=(10, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)

plt.title("Correlation Matrix of Broadband Access Variables")
plt.tight_layout()
plt.show()
```



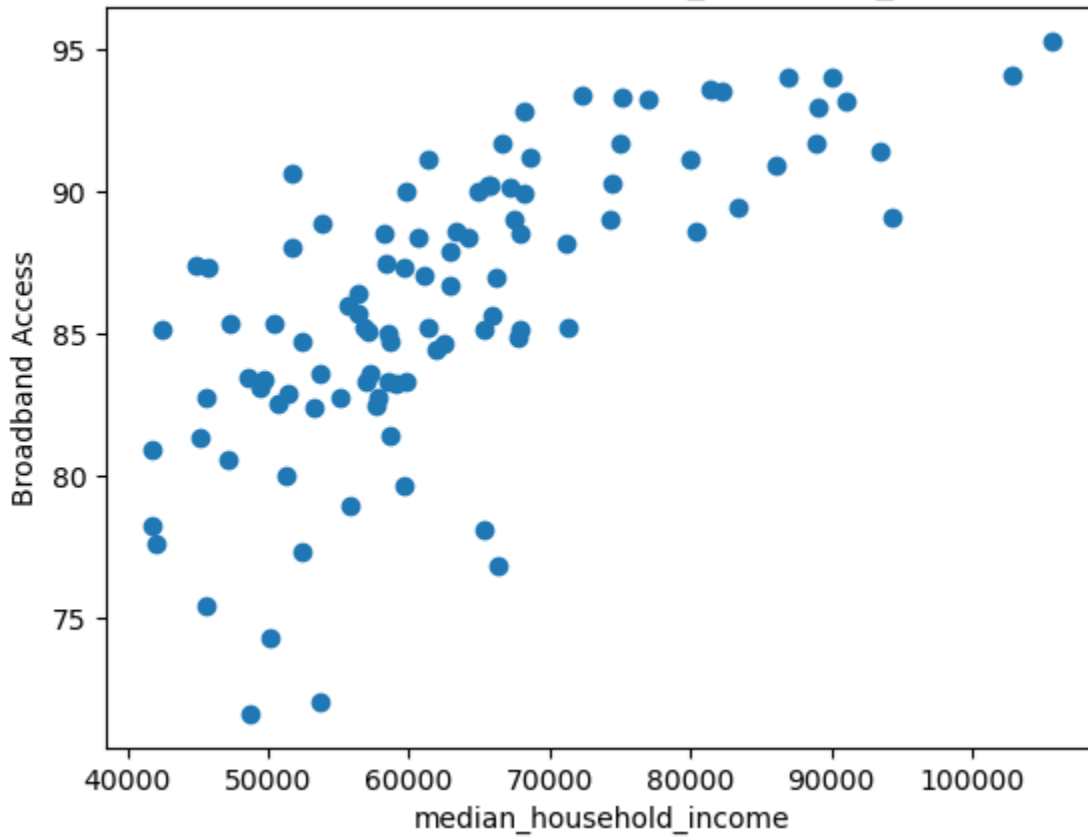
In [471]:

```
#Scatter Plots for Broadband Access vs. Predictors Except County Name
```

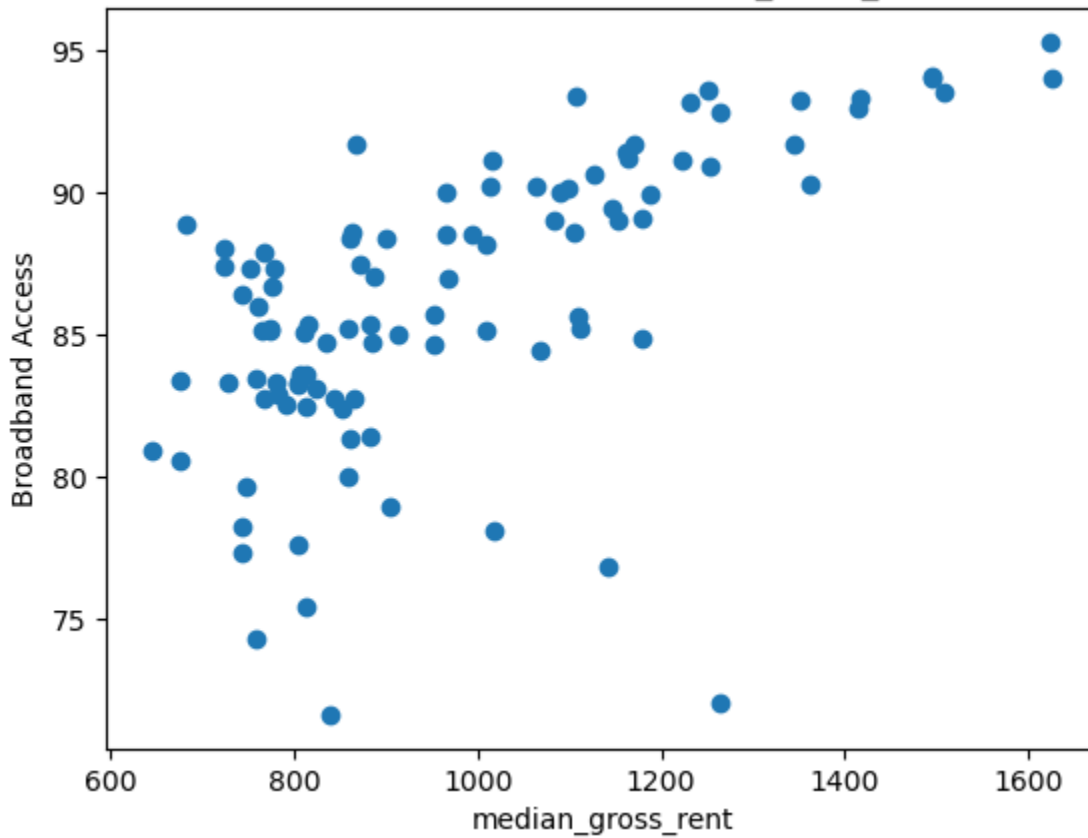
```
predictor_columns = [
    "median_household_income",
    "median_gross_rent",
    "poverty_rate",
    "bachelors_or_higher",
    "unemployment_rate",
    "rural_percent"
]

for predictor in predictor_columns:
    plt.scatter(model_df[predictor], model_df["broadband_access"])
    plt.xlabel(predictor)
    plt.ylabel("Broadband Access")
    plt.title(f"Broadband Access vs. {predictor}")
    plt.show()
```

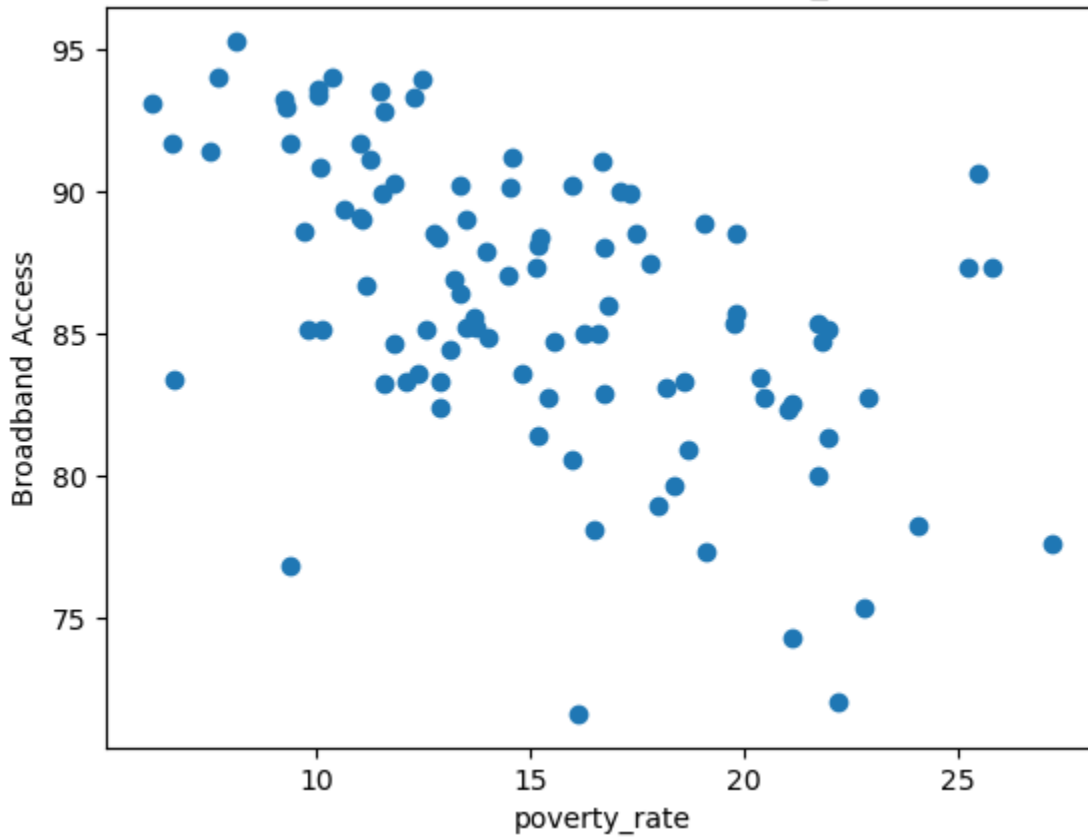
Broadband Access vs. median_household_income



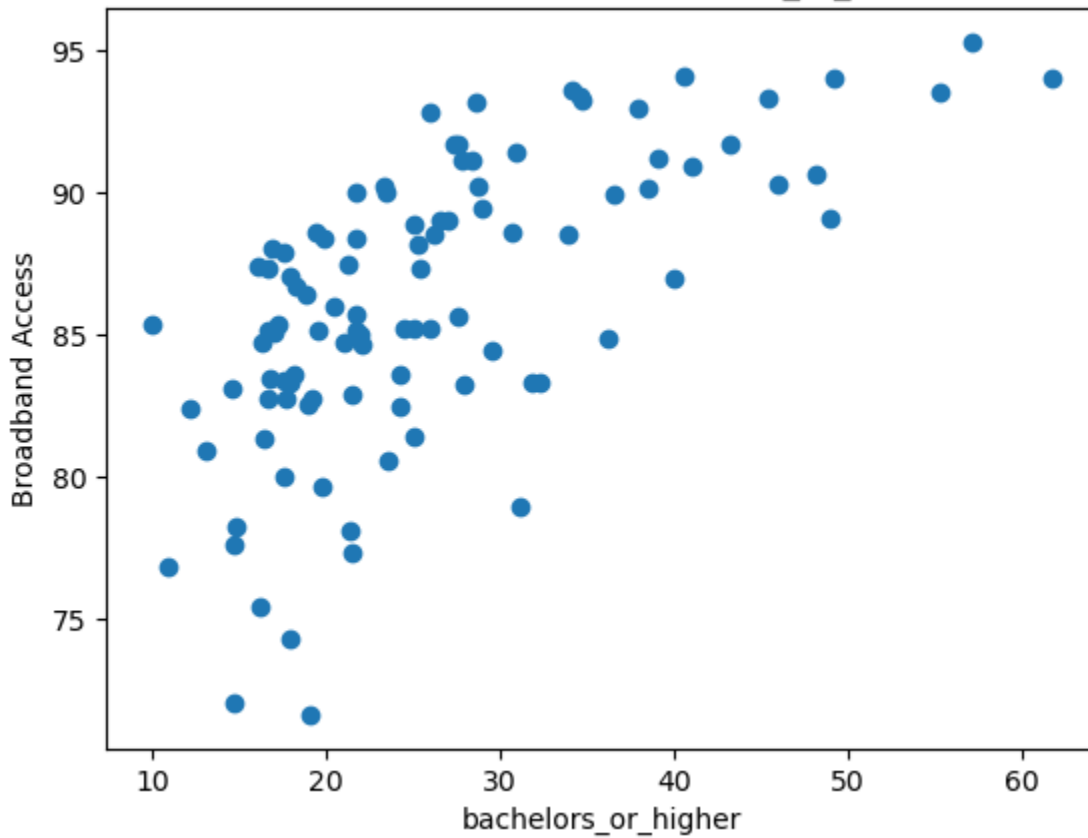
Broadband Access vs. median_gross_rent

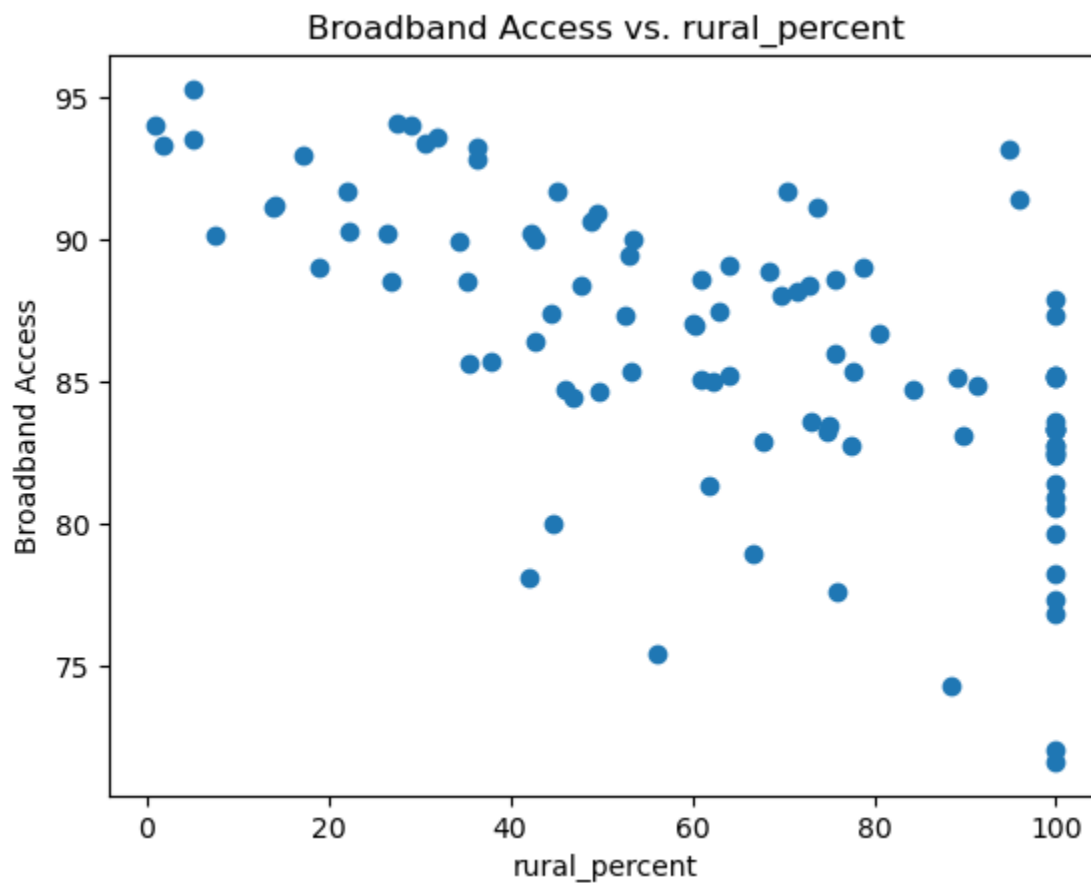
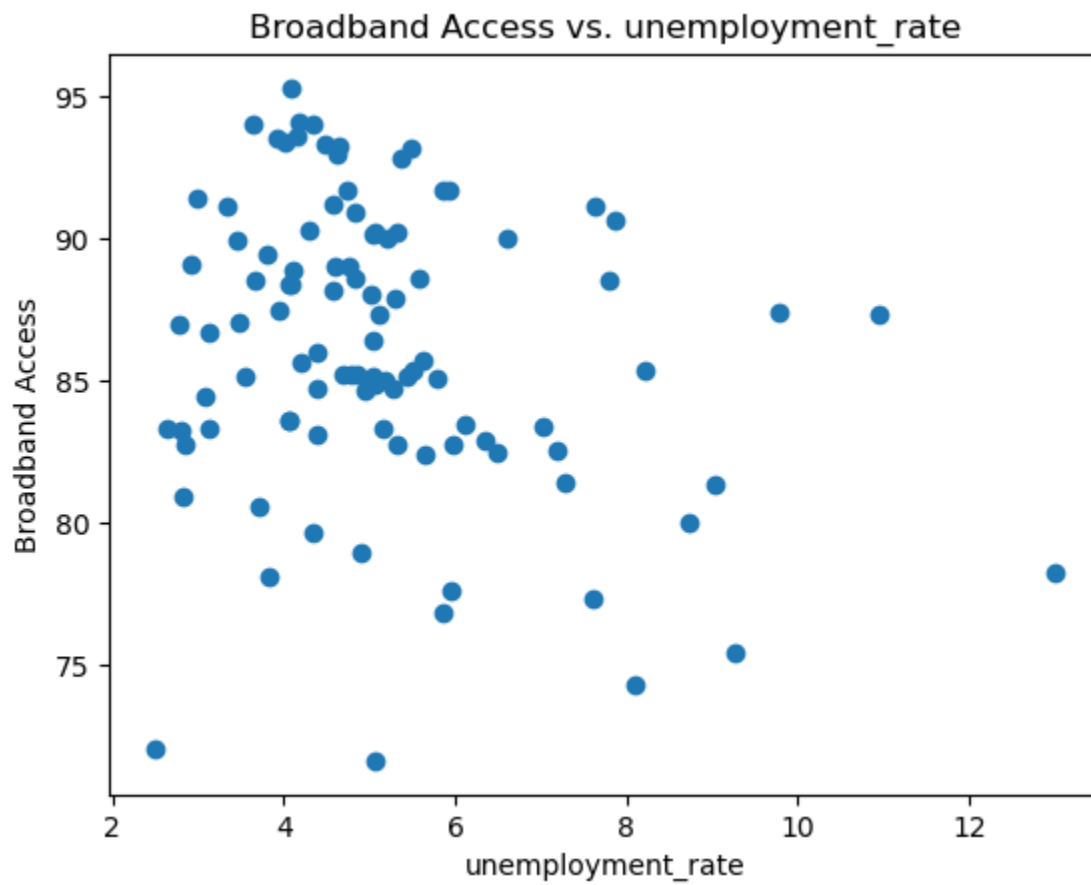


Broadband Access vs. poverty_rate



Broadband Access vs. bachelors_or_higher





In [472]:

```
#Creating Base Regression Model
base_model = smf.ols(
    formula="broadband_access ~ median_household_income + median_gross_rent + poverty_ra
```

```
data=model_df
).fit()

base_model.summary()
```

Out[472]:

OLS Regression Results

Dep. Variable:	broadband_access	R-squared:	0.642
Model:	OLS	Adj. R-squared:	0.619
Method:	Least Squares	F-statistic:	27.80
Date:	Sun, 04 Oct 2026	Prob (F-statistic):	8.70e-19
Time:	12:11:23	Log-Likelihood:	-252.75
No. Observations:	100	AIC:	519.5
Df Residuals:	93	BIC:	537.7
Df Model:	6		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	84.4212	4.617	18.286	0.000	75.253	93.589
median_household_income	0.0002	5.84e-05	3.698	0.000	0.000	0.000
median_gross_rent	-0.0080	0.003	-2.550	0.012	-0.014	-0.002
poverty_rate	-0.0936	0.119	-0.788	0.433	-0.330	0.142
bachelors_or_higher	0.0803	0.052	1.552	0.124	-0.022	0.183
unemployment_rate	0.0986	0.212	0.465	0.643	-0.323	0.520
rural_percent	-0.0827	0.015	-5.343	0.000	-0.113	-0.052

Omnibus:	12.682	Durbin-Watson:	2.027
Prob(Omnibus):	0.002	Jarque-Bera (JB):	13.606
Skew:	-0.781	Prob(JB):	0.00111
Kurtosis:	3.908	Cond. No.	9.51e+05

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 9.51e+05. This might indicate that there are strong multicollinearity or other numerical problems.

In [473]:

```
#Checking VIF (Variance Inflation Factor) for Multicollinearity
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif_data = pd.DataFrame()
vif_data["Feature"] = model_df[['median_household_income', 'median_gross_rent', 'poverty_rate', 'bachelors_or_higher', 'unemployment_rate', 'rural_percent']]
```

```
vif_data["VIF"] = [variance_inflation_factor(model_df[['median_household_income', 'media  
print(vif_data)
```

	Feature	VIF
0	median_household_income	93.031274
1	median_gross_rent	97.711719
2	poverty_rate	18.195290
3	bachelors_or_higher	21.341425
4	unemployment_rate	13.270300
5	rural_percent	8.495871

In [474]:

```
#Training and Testing Split for Model Evaluation
```

```
from sklearn.model_selection import train_test_split
```

```
X = model_df[  
    [  
        "median_household_income",  
        "median_gross_rent",  
        "poverty_rate",  
        "bachelors_or_higher",  
        "unemployment_rate",  
        "rural_percent"  
    ]  
]
```

```
y = model_df["broadband_access"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42  
)
```

```
#Accuracy Metrics for Model Evaluation
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, root_mean
```

```
#Evaluate Model Performance
```

```
y_true = y_test  
y_pred = base_model.predict(X_test)  
#y_pred = new_model_2.predict(X_test)
```

```
base_mae = mean_absolute_error(y_test, y_pred)  
base_rmse = root_mean_squared_error(y_test, y_pred)  
base_r2 = r2_score(y_test, y_pred)
```

```
print("R²:", round(base_r2, 4))  
print("MAE:", round(base_mae, 4))  
print("RMSE:", round(base_rmse, 4))
```

R²: 0.7955
MAE: 1.5107
RMSE: 1.8443

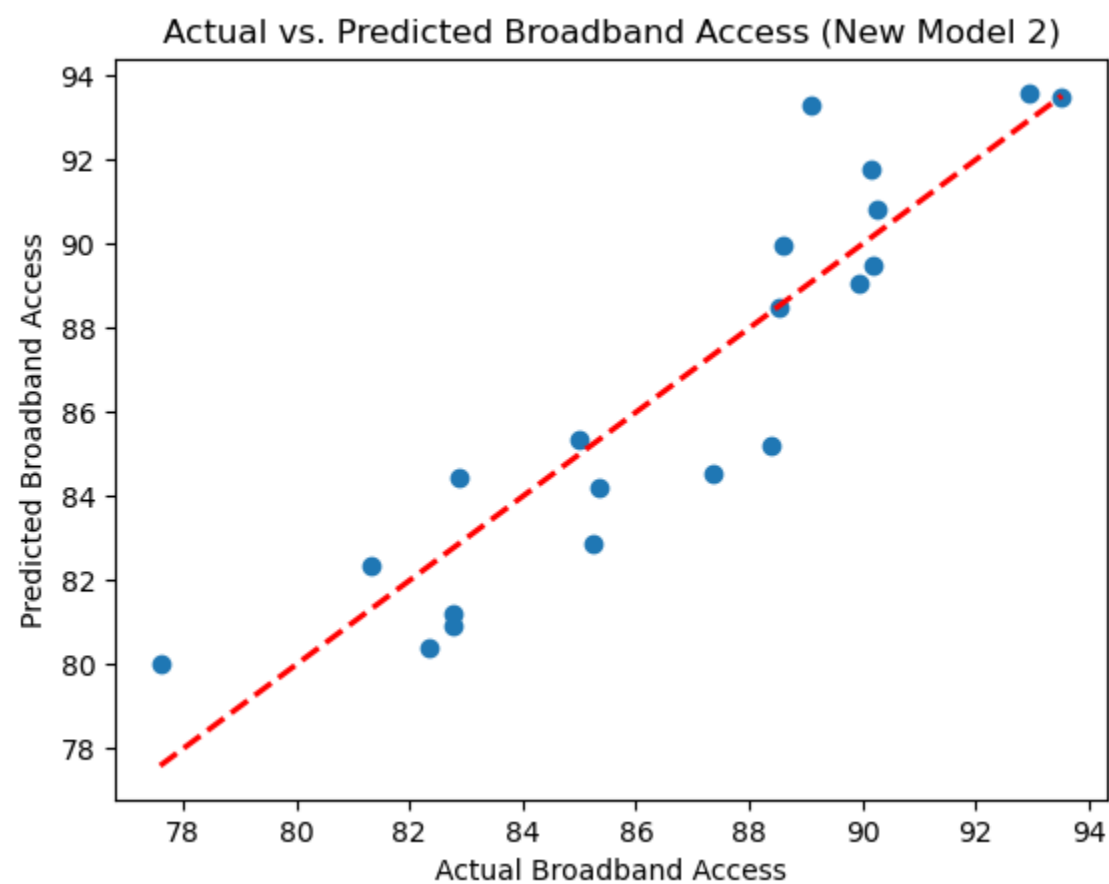
In []:

```
#Actual vs. Predicted Plot for the Base Model
```

```
y_pred = base_model.predict(X_test)
```

```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Broadband Access")
plt.ylabel("Predicted Broadband Access")
plt.title("Actual vs. Predicted Broadband Access (New Model 2)")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2, color=
plt.show()
```

```
/var/folders/8q/_n0ttssj2xqfmkjpr2bvldz40000gn/T/ipykernel_19034/3096484364.py:8: UserWarning: color is redundantly defined by the 'color' keyword argument and the fmt string "k--" (-> color='k'). The keyword argument will take precedence.
  plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2, color='red') # Diagonal line
```



```
In [476]:
#Creating New Regression Model. Removing unemployment_rate mainly due to low significance
new_model = smf.ols(
    formula="broadband_access ~ median_household_income + median_gross_rent + poverty_rate",
    data=model_df
).fit()

new_model.summary()
```

```
Out[476]:
```

OLS Regression Results			
Dep. Variable:	broadband_access	R-squared:	0.641
Model:	OLS	Adj. R-squared:	0.622
Method:	Least Squares	F-statistic:	33.60
Date:	Sun, 04 Oct 2026	Prob (F-statistic):	1.57e-19
Time:	12:11:23	Log-Likelihood:	-252.87

No. Observations:	100	AIC:	517.7
Df Residuals:	94	BIC:	533.4
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	84.7914	4.528	18.724	0.000	75.800	93.783
median_household_income	0.0002	5.81e-05	3.696	0.000	9.94e-05	0.000
median_gross_rent	-0.0079	0.003	-2.540	0.013	-0.014	-0.002
poverty_rate	-0.0753	0.112	-0.675	0.502	-0.297	0.146
bachelors_or_higher	0.0775	0.051	1.515	0.133	-0.024	0.179
rural_percent	-0.0837	0.015	-5.483	0.000	-0.114	-0.053

Omnibus:	12.426	Durbin-Watson:	2.011
Prob(Omnibus):	0.002	Jarque-Bera (JB):	13.290
Skew:	-0.767	Prob(JB):	0.00130
Kurtosis:	3.914	Cond. No.	9.36e+05

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 9.36e+05. This might indicate that there are strong multicollinearity or other numerical problems.

Slightly improved Adj. R-Squared

In [482]:

```
#Training and Testing Split for New Model Evaluation
from sklearn.model_selection import train_test_split

X = model_df[
    [
        "median_household_income",
        "median_gross_rent",
        "poverty_rate",
        "bachelors_or_higher",
        "rural_percent"
    ]
]

y = model_df["broadband_access"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
```

```

)

#Accuracy Metrics for Model Evaluation
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, root_mean

#Evaluate Model Performance
y_true = y_test
y_pred = new_model_2.predict(X_test)

new_model_mae = mean_absolute_error(y_test, y_pred)
new_model_rmse = root_mean_squared_error(y_test, y_pred)
new_model_r2 = r2_score(y_test, y_pred)

print("R²:", round(new_model_r2, 4))
print("MAE:", round(new_model_mae, 4))
print("RMSE:", round(new_model_rmse, 4))

```

R²: 0.7833
MAE: 1.5115
RMSE: 1.8988

In [478]:

```

#New Regression Model. Also removing Poverty Rate mainly due to low significance (p-value)
new_model_2 = smf.ols(
    formula="broadband_access ~ median_household_income + median_gross_rent + bachelors_
    data=model_df
).fit()

new_model_2.summary()

```

Out[478]:

OLS Regression Results							
Dep. Variable:	broadband_access			R-squared:	0.639		
Model:	OLS		Adj. R-squared:	0.624			
Method:	Least Squares			F-statistic:	42.13		
Date:	Sun, 04 Oct 2026		Prob (F-statistic):	2.82e-20			
Time:	12:11:23		Log-Likelihood:	-253.11			
No. Observations:	100			AIC:	516.2		
Df Residuals:	95			BIC:	529.2		
Df Model:	4						
Covariance Type:	nonrobust						
		coef	std err	t	P> t	[0.025	0.975]
Intercept	82.2850	2.582	31.875	0.000	77.160	87.410	
median_household_income	0.0002	4.21e-05	5.744	0.000	0.000	0.000	
median_gross_rent	-0.0083	0.003	-2.746	0.007	-0.014	-0.002	
bachelors_or_higher	0.0780	0.051	1.528	0.130	-0.023	0.179	
rural_percent	-0.0826	0.015	-5.458	0.000	-0.113	-0.053	
Omnibus:	13.473	Durbin-Watson:	2.024				

Prob(Omnibus):	0.001	Jarque-Bera (JB):	14.733
Skew:	-0.810	Prob(JB):	0.000632
Kurtosis:	3.956	Cond. No.	5.35e+05

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 5.35e+05. This might indicate that there are strong multicollinearity or other numerical problems.

In [479]:

```
#Training and Testing Split for New Model 2 Evaluation
from sklearn.model_selection import train_test_split

X = model_df[
    [
        "median_household_income",
        "median_gross_rent",
        "bachelors_or_higher",
        "rural_percent"
    ]
]

y = model_df["broadband_access"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

#Accuracy Metrics for Model Evaluation
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, root_mean

#Evaluate Model Performance
y_true = y_test
y_pred = new_model_2.predict(X_test)

model_2_mae = mean_absolute_error(y_test, y_pred)
model_2_rmse = root_mean_squared_error(y_test, y_pred)
model_2_r2 = r2_score(y_test, y_pred)

print("R²:", round(model_2_r2, 4))
print("MAE:", round(model_2_mae, 4))
print("RMSE:", round(model_2_rmse, 4))
```

R²: 0.7833

MAE: 1.5115

RMSE: 1.8988

In [480]:

```
#Comparison of Model Performance Metrics
performance_metrics = pd.DataFrame({
    "Model": ["Base Model", "New Model", "New Model 2"],
```

```

    "R²": [base_r2, new_model_r2, model_2_r2],
    "MAE": [base_mae, new_model_mae, model_2_mae],
    "RMSE": [base_rmse, new_model_rmse, model_2_rmse]
})
print(performance_metrics)

```

	Model	R ²	MAE	RMSE
0	Base Model	0.795528	1.510715	1.844325
1	New Model	0.783272	1.511484	1.898797
2	New Model 2	0.783272	1.511484	1.898797

In [493]:

```

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

X = model_df[
    [
        "median_household_income",
        "median_gross_rent",
        "poverty_rate",
        "bachelors_or_higher",
        "unemployment_rate",
        "rural_percent"
    ]
]

y = model_df["broadband_access"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

tree_model = DecisionTreeRegressor(
    max_depth=5,
    random_state=42
)

tree_model.fit(X_train, y_train)

from sklearn.metrics import (
    r2_score,
    mean_absolute_error,
    root_mean_squared_error
)

y_pred_tree = tree_model.predict(X_test)
tree_r2 = r2_score(y_test, y_pred_tree)
tree_mae = mean_absolute_error(y_test, y_pred_tree)
tree_rmse = root_mean_squared_error(y_test, y_pred_tree)

print("Decision Tree R²:", tree_r2)
print("Decision Tree MAE:", tree_mae)
print("Decision Tree RMSE:", tree_rmse)

```

Decision Tree R²: -1.217813241810525
 Decision Tree MAE: 4.689244101215953

Decision Tree RMSE: 6.074118365024803

In [495]:

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
```

```
plt.figure(figsize=(18, 9))
```

```
plot_tree(
    tree_model,
    max_depth=3,
    feature_names=X.columns,
    filled=True,
    rounded=True,
    precision=1,
    fontsize=9
)
```

```
plt.title("Decision Tree Regression: Predicting Broadband Access")
plt.tight_layout()
plt.show()
```

